Adaptive Difficulty in Games: Leveraging Reinforcement Learning for Player-Driven Immersion

Aditya V. Singh, Ankit Gundewar, Shishir Kallapur, and Ghasif Syed

Northeastern University, Boston
{singh.adityav,gundewar.a,kallapur.shi,syed.g}@northeastern.edu

1 Abstract

We propose a project that aims to use Reinforcement Learning (RL) to create games that adapt to each player's unique strengths, weaknesses, and preferences of players. We also propose an Adversarial RL system to test both "player" and "environment" working together, where the player is trying to clear levels with ease whereas the "environment" is continuously learning the "player"'s skills to make levels more difficult. The help-fullness of this system is twofold, one being making the "player" agent robust against any environment and second is to automate game testing without manual labour.

2 Introduction

In the vibrant landscape of modern gaming, the pursuit of immersiveness stands as a cornerstone in captivating players within virtual worlds, fostering emotional connections, and engendering memorable experiences. Games, as interactive mediums, have continually evolved to offer increasingly immersive environments, blurring the boundaries between reality and the digital realm. However, a persistent challenge looms: the inherent difficulty in crafting experiences that resonate deeply with each player's individuality.Amidst this quest for immersion, the one-size-fits-all approach to game difficulty often falls short.Recognizing this discrepancy, this paper seeks to harness the power of Reinforcement Learning (RL) to dynamically calibrate game difficulty levels, tailoring them to the nuanced skill levels and preferences of individual players.

Instead of the usual "player" learning to traverse a fixed environment, we propose to making the environment an agent to learn the player's moves and skill. Pitting a regular player agent against this can make an automated game testing environment to scope out any game breaking bugs.The motivation behind this project was to give every player a unique experience by leveraging this "learned" environment with Procedural Content Generation to create a unique experience every time.

3 Related Work

This projects biggest inspiration was a recent paper by SEED at Electronic Arts. presented at the IEEE Conference on Games 2021[1]. We have tried to build on their idea of ever-evolving environments to fully flexible environment with complete liberty of the hyper parameters of level generation. Recent studies on modern 3D games have shown the potential of Reinforcement Learning to be used to develop Game-AI[2]. There has been previous exploration on RL agents learning to play games showing that agents start "memorizing" the fixed environments[3]. [4] explores RL to train level-generating agents 'by converting 2D environments to Markov-Decision-Processes.

4 Methodology

4.1 The Game

We have used python and the pygame library to develop our own simple game so that there is full control and access to all game variables and actions. It is a platformer game, where the goal is to jump up vertically to reach a certain score. Figure 1 shows a screenshot of the game. The game has two main entities:

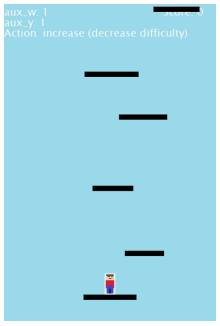

Fig. 1. The platformer game.

- The player: It has two actions: Left or Right. The player automatically jumps if its on a platform when left or right is pressed.
- The platforms: The platforms are generated with variable width and height between each one. This variability is controlled by hyper parameters.

4.2 Q-Learning and Deep Q Learning

Q-learning is a model-free reinforcement learning algorithm used to find the optimal action-selection policy for a Markov decision process (MDP) by learning

the quality of actions in a given state. The algorithm learns an action-value function Q(s,a), representing the expected cumulative future reward when taking action a in state s. The core update rule of Q-learning is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \cdot (reward + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a))$$

where, $Q(s,a)$ is the current estimate of action value of state s and action a.
α is the learning rate.
γ is the discount factor to determine how previous actions influence future ones.
a' and s' are the future action and state.

Deep Q Learning: While the q-learning algorithm is a powerful tool for agents to learn in fully observable environments with discrete state spaces, it fails to work for something like a game where there can be unobserved states which are continuous. Q-learning works by maintaining a state action Q Table which consists cells for all state-action pairs. These cells are constantly updated by the equation above. Maintaining such a Q table, where new unseen continuous states are added at every step leads to an infinitely large table, leading to memory and efficiency issues. Here is where Deep Q Learning comes to the rescue.
Deep Q-learning extends Q-learning by using a deep neural network to approximate the action-value function, enabling it to handle high-dimensional state spaces. Here's a concise technical summary of Deep Q-learning:

Deep Q-learning employs a neural network, $Q(s,a;\theta)$, parameterized by weights θ, to approximate the action-value function $Q(s,a)$. The network takes a state s as input and outputs Q-values for each possible action a.

The objective is to minimize the temporal difference (TD) error between the predicted Q-values and the target Q-values:

$$Loss(\theta) = E[(reward + \gamma \cdot \max_{a'} Q(s',a';\theta^-) - Q(s,a;\theta))^2]$$

where, θ^- represents the parameters of a target network used to stabilize learning.
The neural network is updated by minimizing this loss function using stochastic gradient descent or its variants. The process involves iteratively updating the neural network's weights to minimize the difference between predicted and target Q-values. Through repeated interactions with the environment and learning from stored "experiences", the neural network converges towards approximating the optimal action-value function, enabling efficient and effective decision-making in complex environments. We store "experiences" in memory to enhance the efficiency and stability of learning by storing and reusing past experiences during training. It involves storing transitions (state, action, reward, next state) observed during interaction with the environment in a replay buffer. Figure 2 shows a general architecture of a Deep Q Network.

4.3 The Platform agent

The RL model learns through observations, actions and the rewards after every episode of the game. Figure 3 shows the block diagram of the system loop.

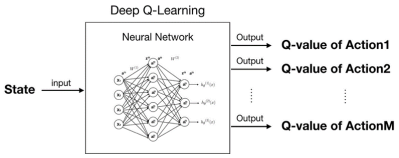

Fig. 2. DQN

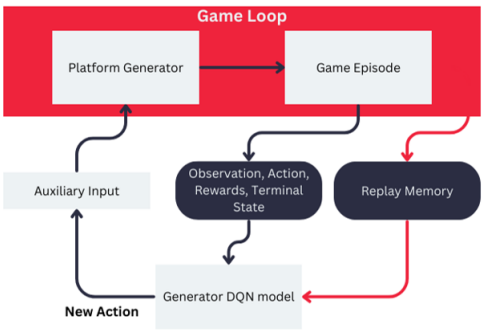

Fig. 3. The platform agent training loop

Auxiliary Inputs: These are the hyper parameters responsible for generating the actual platforms. These control the range of variability in the generated platforms. For the game, there are two inputs controlling the width and distance between each respectively. These inputs are also the state space for our RL model. The model learns to predict actions to increase or decrease these inputs values for the next level. Increasing these values generates wider and closer levels, thus decreasing difficulty, and vice versa. Fig 4 and 5 show the platforms generated at high and low values of the auxiliary inputs.

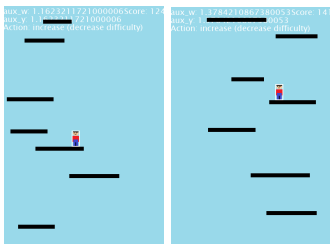

Fig. 4. The platforms at high auxiliary values (low difficulty).

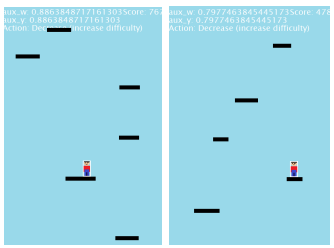

Fig. 5. The platforms at low auxiliary values (high difficulty).

Generator model setup: Table 1 shows the basic generator setup with its discount factor γ

Observations	[aux_y, aux_w]
Actions	[Increase, Decrease]
γ	0.9

Reward function: We have implemented a custom reward function taking into consideration the score achieved by the player and the time taken by the player to complete a level.
If a player fails the level then the reward is calculated as follows:

$$reward = -0.5 \cdot (current\ score - mean\ of\ last\ 5\ scores)$$

If a player completes the level then the reward is calculated as follows:

$$reward = 0.5 \cdot (current\ time\ taken - mean\ of\ last\ 5\ time\ periods)$$

4.4 The Player RL:

This is the more standard RL agent of the player learning to traverse through environments. Each frame of the game is stored in the Replay memory and is used as observations for learning. Figure 6 shows the block diagram of the player system loop and Table 2 gives the player model setup. The reward function is simple for this agent; +200 if the current score is higher than previous time-step score, -10 if they are same and -100 if its lower. The small -10 encourages the agent to continuously strive for higher scores or fail immediately.

4.5 Adverse Game Loop:

Finally, we incorporated the 2 models above into a single game loop (Figure 7). The two agents do not have information of each others existence, but the reward signals are sort of a direct communication between the two for learning, i.e., player agent's reward signal directly affects the platform agent's reward signal.

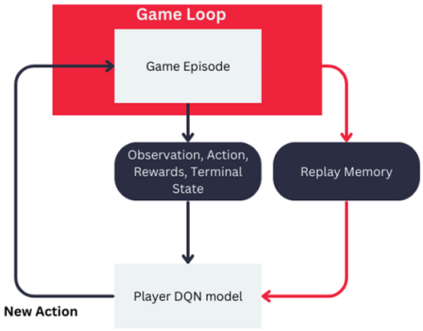

Fig. 6. The player agent training loop.

Observations	Screenshot of the current game screen reduced to (50,50,1) size
Actions	Left, Right
γ	0.9

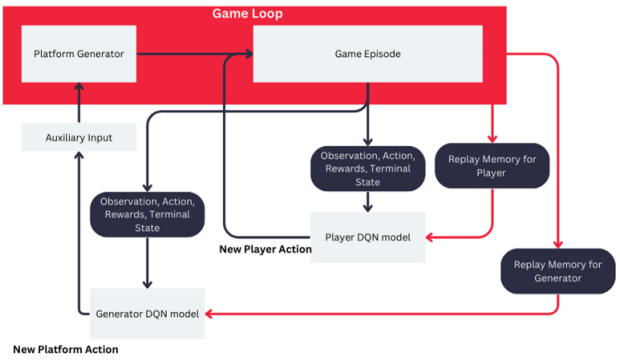

Fig. 7. The player agent training loop.

5 Experiments and Results

We setup 5 experiments in total, out of which 4 were fully automated tests. The first setup was just the platform generator learning through a human player's inputs. This is obviously not viable to train for hours as there is requirement of the user to play for numerous hours (hence one of the motivations behind the adverse game). For the player agent, we ran 3 experiments: one where the platforms are completely random, and the other 2 with different fixed orientations of the platforms. In all these tests, the agent was able to achieve consistently higher scores with each time step after a thousand iterations (Figure 8).

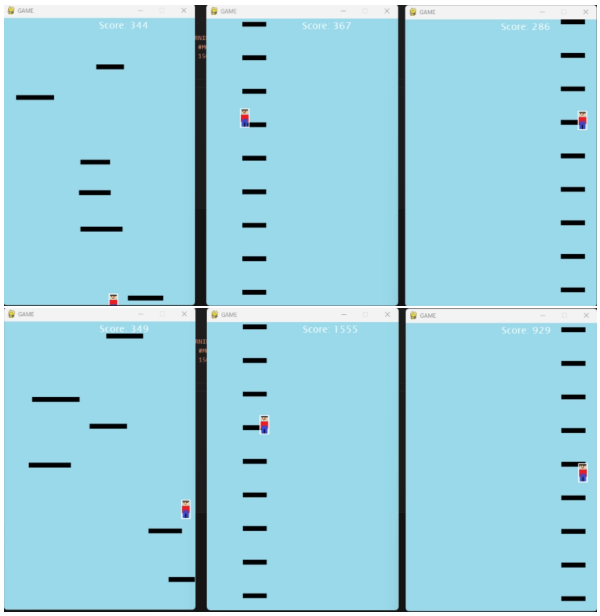

Fig. 8. Scores at 2 consecutive levels.

Unfortunately, while training the adverse system, we quickly ran into Out-of-Memory issues while using TensorFlow to setup our DQN model. With 2 replay buffers working simultaneously, after first few minutes, the game itself becomes very slow, leading up to OOM issues.

6 Discussion and Conclusions

We have proposed 2 systems, the environment system to make a game more immersive, and the adverse system to make more robust player agent and game testing. The immediate plan of action for the continuation of this project is to try train the adverse system on powerful systems and tune it to deal with

the OOM issues. Once that is fixed, we can map the rewards and make more complex reward functions to help with training. Some premature solutions for OOM that have come up in discussion are shifting our model to the PyTorch library and parallelization of training. Shifting to a different language such as C# is also something which may help in efficiency and deployment in diverse environments.

Future scope of this work is to scale it to larger and more complex environments, with large observation spaces and complex actions.

7 Team Contribution

All of the work was done in collaboration with regular ideation/discussions. Ghasif worked on pygame game implementation with inputs from Aditya. Ankit and Shishir worked on player and platform DQN models. Aditya worked on implementation of the adverse system, with inputs for the DQN models. The project report and presentation was done together in collaboration.

8 Source Code

Github link to source code.

References

1. L. Gisslén, A. Eakins, C. Gordillo, J. Bergdahl and K. Tollmar, "Adversarial Reinforcement Learning for Procedural Content Generation," 2021 IEEE Conference on Games (CoG), Copenhagen, Denmark, 2021, pp. 1-8, doi: 10.1109/CoG52621.2021.9619053.
2. J. Harmer, L. Gisslén, J. del Val, H. Holst, J. Bergdahl, T. Olsson, K. Sjöö, and M. Nordin, "Imitation learning with concurrent actions in 3d games," in 2018 IEEE Conference on Computational Intelligence and Games (CIG), 2018, pp. 1–8.
3. S. Risi and J. Togelius, "Increasing generality in machine learning through procedural content generation," Nature Machine Intelligence, pp. 1–9, 2020.
4. A. Khalifa, P. Bontrager, S. Earle, and J. Togelius, "Pcgrl: Procedural content generation via reinforcement learning," arXiv preprint arXiv:2001.09212, 2020.
5. OpenAI. (Year). GPT-3 [Model]. https://www.openai.com/research/gpt-3